The tricky part is that `BridgeComponent` has two required parameters to be instantiated: `name` and `bridgeDelegate`. Hence, these parameters appear between the *second* set of parentheses. They're passed from the `ButtonComponent`'s initializer in the *first* set of parentheses. This means that when a `ButtonComponent` is created, it will automatically pass those parameters to its superclass, `BridgeComponent`.

Finally, adding `private val` in front of the `bridgeDelegate` property creates an immutable instance variable for us to access *inside* of `ButtonComponent`. You'll use this to grab a reference to the `Fragment` that's rendering the component.

In Ruby, this could look like the following:

```
class ButtonComponent < BridgeComponent
  attr_reader :bridge_delegate
  private :bridge_delegate

  def initialize(name, bridge_delegate)
    @bridge_delegate = bridge_delegate
    super(name, bridge_delegate)
  end
end

class BridgeComponent
  def initialize(name, bridge_delegate)
    # ...
  end
end
```

You might have noticed that this code isn't compiling yet. Move your cursor over the red squiggled `ButtonComponent` and press ⌥↩. Click Implement members and then click OK. This will add an unimplemented, but required, `onReceive(message: Message)` function. I bet you can tell what that's used for—you used a function of the same name in the previous chapter, on iOS, to handle received messages received over the bridge.